

Self-Compressing Vision Tower for Efficient Dense Prediction Tasks

Aditya Shourya ^a

a.shourya@student.maastrichtuniversity.nl

Guangzhi Tang ^a

guangzhi.tang@maastrichtuniversity.nl

Chang Sun ^{a,b}

chang.sun@maastrichtuniversity.nl

^a Institute of Data Science, Faculty of Science and Engineering, Maastricht University, Maastricht, The Netherlands

^b Department of Advanced Computing Sciences, Faculty of Science and Engineering, Maastricht University, Maastricht, The Netherlands

High-resolution dense prediction, essential for tasks such as classification, segmentation, and optical flow, requires models that balance accuracy with efficiency. Most state-of-the-art architectures rely on deep sequential operations that are computationally expensive and difficult to deploy on resource-constrained devices. We introduce a novel, self-compressing vision architecture that integrates structured pruning and quantization across key modules: convolutional layers, transposed convolutions, and linear attention in proportion to their computational cost. By strategically reducing precision and pruning tensors in less critical layers, our method achieves significant model compression with minimal impact on accuracy. We evaluated our approach on fine-grained classification (CUB-200-2011, Country211), semantic segmentation (ADE20K), and optical flow (HD1K). Our models consistently match the accuracy of the current state-of-the-art, EfficientViT, while often reducing both storage requirements and FLOPs. Furthermore, our method outperforms DiffQ in both accuracy and efficiency under identical parameter constraints. We conclude that compression can be a constructive tool for not only reducing model size but also enabling the inspection of module importance across layers, which facilitates better model design for specific target devices.

Date : August 20, 2025

URL :  <https://github.com/adishourya/SelfcompressingDepthWiseAttn>



1. Introduction

High-resolution dense prediction remains a critical task in computer vision. The effectiveness of a vision model is often evaluated by its performance both in accuracy and latency on large-scale dense prediction benchmarks.

Low-compute vision models enable real-time applications such as autonomous drones, on-device image processing, and computational photography. In domains like sports or wildlife photography, throughput can be just as important as accuracy. However, most state-of-the-art vision models rely on deep, sequential architectures, which are unsuitable for

deployment on consumer-grade devices with limited computational resources. In addition to their role in standalone vision tasks, vision towers that perform well on dense prediction problems are also considered as candidates for the visual encoders in vision-language models specially in small Vision Language Models [1].

Recent research suggest that the deeper layers of models may contribute less to overall accuracy than earlier layers, which indicates over-parameterization of networks and challenges the need for high computational cost in deeper layers [2], [3]. For instance, MiniMax-01 [4] employs full softmax attention ($O(N^2)$) only in few initial layers, while adopting

more compute efficient alternatives such as linear attention ($O(N)$) in the deeper layers to reduce total compute requirements.

Inspired by these insights, we propose a fully prunable and quantizable vision model as a backbone where all major learnable components such as convolution and attention weights and linear layers can be compressed and pruned to improve model storage efficiency and improve accuracy during low precision inference for dense prediction tasks.

A key hypothesis we examine in this work is the need for uniform allocation of model precision and compute cost across model depth. In a non self-compressing models [5], [6], weight precision and compute budgets are assigned identically to all layers regardless of their relative contribution to final performance. This uniform treatment can result in overparameterization and inefficient use of compute resources. We extensively evaluate our model on image classification, segmentation and optical flow problems. Our model matches accuracy over existing state of the art method. and present the following key contribution:

1. **Unified Compression Strategy:** We propose a unified framework that to quantize and compress the weights of modules proportionally to their computational cost, enabling consistent compression across all major learnable components Convolutional, Attention Weights, and Linear Layers via pruning and quantization.
2. **Efficient Deployment :** Our compressed model sustains accuracy levels comparable to the FP32 baseline during FP8 inference, while significantly improving throughput during inference.

2. Related Study

Our methodology builds upon recent advancements in efficient vision towers and quantization approaches, drawing insights from prior work. Below, we summarize key contributions from related works that have informed our approach.

1. **EfficientViT [7]** carefully selects computational modules to maximize accuracy while maintaining

high throughput. The architecture strategically offloads local inductive bias computation to light-weight convolution kernels, avoiding the computationally expensive full softmax attention. For example, it employs grouped depthwise convolutions to capture rich local structure at low cost, while using linear attention to obtain global receptive fields.

This careful allocation of operations results in significant speedups while maintaining state-of-the-art accuracy across dense prediction tasks. However, EfficientViT focuses on module selection and efficient operations without incorporating compression strategies, which could further improve both throughput and accuracy while reducing model storage cost.

2. **LayerDrop [8]** applies structured, layer-wise dropout in Transformer networks. During training, entire layers are randomly skipped either stochastically or through learned skipping which regularizes the network and makes it robust to layer removal. At inference, this enables extraction of arbitrary-depth sub-networks from a single model without retraining, providing scalability across compute budgets. While effective for pruning at the layer level, LayerDrop does not perform fine-grained compression such as bit-depth quantization or weight-level pruning.
3. **DiffQ [9]** presents a differentiable quantization framework that learns both weights and their bit depths without gradient approximations. To emulate the effect of quantization during training, it injects independent pseudo-quantization noise, enabling end-to-end optimization of precision at the level of individual weights or small groups, governed by a single accuracy-size trade-off hyperparameter. This design achieves competitive compression across vision, language, and audio tasks.
4. **Self-Compressing Networks [10]** remove the need for noise emulation by directly learning bit depths as trainable parameters, which can be reduced to zero to prune weights or entire channels. Unlike DiffQ's coalesced memory grouping, they adopt coarser mode-level (channel) grouping for

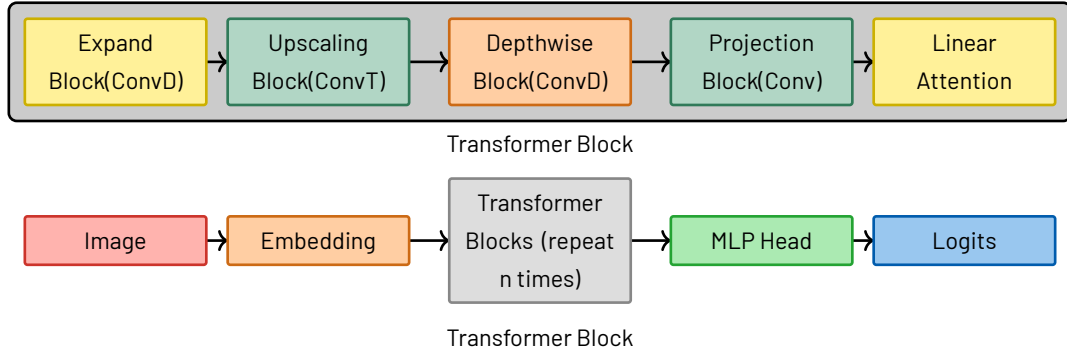


Figure 1: **Model architecture.** Each transformer block consists of an expansion stage using depthwise convolution (ConvD), an upscaling stage using transposed convolution (ConvT), another depthwise convolution, a projection stage, and a lightweight linear attention module. The output is passed through an MLP head to produce logits for the dense prediction task.

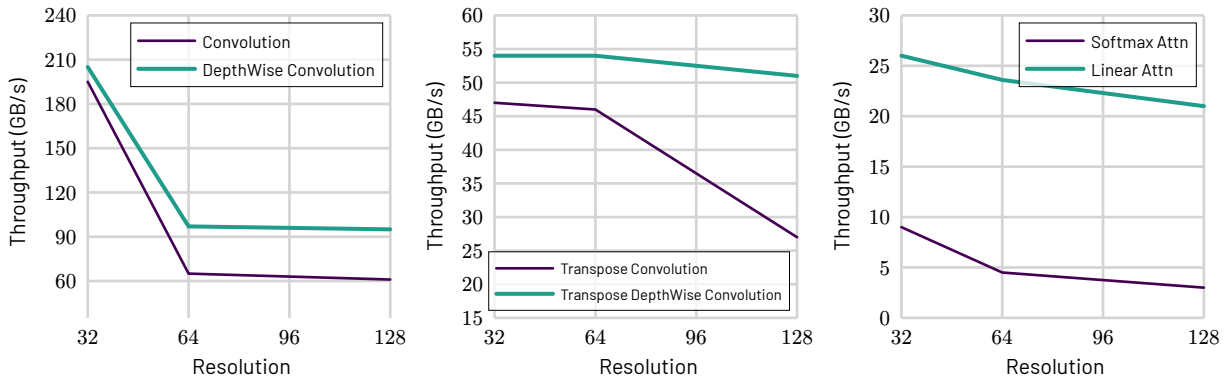


Figure 2: **Module Throughput:** Throughput evaluation of convolutional, transposed convolutional, depthwise convolutional, and attention-based modules across varying input resolutions on an NVIDIA 4070 [11]. The results demonstrate that depthwise operations and linear attention sustain significantly higher throughput as resolution increases, reflecting their computational efficiency compared to their ungrouped or softmax-based counterparts.

greater stability and reduced forgetting. They also treat number format parameters (scale, exponent, bit depth) as optimizable network parameters, allowing complete channel elimination without specialized hardware, and rely on the Straight-Through Estimator instead of pseudo-noise for gradient flow.

3. Method

We design our model by adapting ideas from recent vision models [7] and [12]. Our architecture follows a decoder-only transformer structure. The input image is first projected into a latent sequence of embeddings, where each embedding dimension corresponds to a learnable convolution kernel. Each transformer block is then implemented as a sequential composition of convolutional and transpose convolution operations [13], consisting of channel expansion (convolution), learned upscaling (grouped transposed convolution), depthwise convolution, and channel

projection (convolution). After these local operations, a linear attention module is applied to capture global receptive fields as in Figure 1.

We quantize all matrices associated to the compute modules as the primary method to quantize and prune modes of weights. Specifically, the penalty of operations is designed to be proportional to the size of the parameter matrices and inversely proportional to the measured throughput on target device for each module. This penalty is incorporated into the training objective as a hinge-style regularizer, resulting in the overall loss function where L_o is the original loss of the model and γ acts as the compression factor over the module size normalizing constant C as in Equation 1.

$$\text{Loss} = L_o + \frac{\gamma}{C} \sum_{k=1}^{\text{all modules}} \alpha_k \text{size}_k \quad (1)$$

For, our experiments use quantization as used in [10] to quantize weights in the forward pass and use symmetric differentiable number Q8A format [14] given in

Equation 2. The bit-depth b and the scaling factor s is shared per mode of parameters in W .

$$Q(W, b, s)_k = 2^e \lfloor \text{clip}\left(\frac{W}{2^s}, -2^{b-1}, 2^{b-1} - 1\right) \rfloor \quad (2)$$

The $\lfloor \cdot \rfloor$ acts as a straight through estimator [15] for the rounding function which returns the identity of upstream gradient during backward pass.

In the following sections, we use similar methods from [10] and derive the penalty cost for modules in our transformer block.

1. **Convolution:** are most frequent operation in our transformer block because they provide a strong local inductive bias. While full softmax attention [16] can capture both local patterns and long-range dependencies, its computational cost is much higher. Attention over an image of height H and width W , with patch size p and embedding dimension d , scales as $O((HW)^2 dp^{-2})$, making it quadratic in the spatial resolution. In contrast, a convolutional kernel of shape (O, I, H, W) , with kernel size k , has cost $O(OIHWk^2)$.

We further optimize the convolution operation through quantization. Each output channel O_i is tied to a learnable bit-depth parameter $b_{\{i,l\}}$ and an exponent parameter $e_{\{i,l\}}$. The quantized kernel weights are obtained by scaling the raw kernel as in Equation 2. To formalize the compression cost, we define the effective size of layer l as

$$\text{size}(l) = IHW \sum_{i=1}^O \max(0, \lceil b_{i,l} \rceil) \quad (3)$$

This penalizes convolution module with higher bit allocations per output mode, pushing the weight towards more sparse filters in the filter bank. To align compression with runtime efficiency, we scale this penalty λ_l inversely with the measured throughput T_l on the target device of each layer. Faster modules thus incur a smaller penalty, ensuring the optimization objective favors both compactness and speed:

$$\lambda_l = \frac{1}{T_l} \quad (4)$$

$$L_{\text{pen},l} = \lambda_l \text{size}(l) \quad (5)$$

2. **Depthwise Convolution:** One way to improve throughput in convolutional layers is to apply strided convolution, which increases throughput linearly with spatial resolution, though at the expense of reducing the effective local receptive field. Another option is grouped convolution, where the input channels are divided into g groups. In this case, the convolutional kernel has shape $(O, \frac{I}{g}, k^2)$, with g denoting the number of groups. A special case of this is depthwise convolution, where the number of groups g equals the number of input modes I . As shown in Figure 2, depthwise convolution [12] has a higher throughput than ungrouped convolution when spatial resolution increases.

In our design, we employ depthwise convolutions in the expansion phase of the vision tower. This ensures that the computational cost does not scale with the embedding dimension, unlike ungrouped convolutions. To regularize these layers, we apply the same heuristic penalty formulation as in the ungrouped case, thereby encouraging efficient yet expressive expansion blocks.

3. **Transposed Convolution:** For the upscaling stage, we employ transposed convolution after the projection block, inspired by [17], to generate upscaled features from expanded convolution outputs. Although transposed convolutions are more expensive than standard convolutions (see Figure 2), we opt for ungrouped transposed operation since they provide richer local features than depthwise alternatives. To mitigate cost, we restrict the number of output channels rather than performing full ungrouped upscaling, and we apply a stronger penalty to the module size. This strategy follows the bottleneck principle in ResNet [18] and the reduced-channel decoding of U-Net [19], where only a limited set of upscaled features is sufficient to recover fine spatial details. In the subsequent stages, we apply one final composition of depthwise convolution and strided convolution to efficiently downscale the features for linear attention module.
4. **Linear Attention:** While softmax attention captures both local and global dependencies, local

context is already handled by composition of convolution functions in the previous stages of our transformer block. The linear attention module therefore focuses exclusively on modeling global receptive fields. Unlike standard attention, which scales quadratically with spatial resolution, linear attention computes global interactions in time linear with respect to spatial resolution, following [20] and [21].

We leverage per-head quantization of queries, keys, values, and outputs, which can be broadcasted prior to computation. This design enables integration with Flash Linear Attention [22], improving runtime throughput while preserving linear complexity in token count.

Our implementation of the linear attention module reshapes spatial feature maps into a token sequence, applies the linear attention, and restores the original tensor layout. A residual connection with the learned output tensor is applied, followed by group normalization to ensure numerical stability.

4. Experiments

4.0.1. Setup

Datasets: We evaluate our model on three standard dense prediction tasks **classification**, **segmentation**, and **optical flow** problems.

1. For **classification**, we use CUB-200-2011 [23] and Country211 [24]. CUB-200-2011 contains 11,788 images (~6k train, 6k test) of 200 bird species, collected from online repositories between 2009–2011, with an average resolution around 500×500 pixels and fine-grained species labels. Country211 is a geo-location dataset derived from YFCC100M [25], constructed by sampling 150 training, 50 validation, and 100 test images per country for 211 countries (~63k total). The raw images are drawn from Wikimedia Commons [26] and other Creative Commons sources that vary in resolution; we resize them to 448×448 pixels for training. Together, these datasets test models both fine-grained

species recognition (local biases) and global geographic locations (global biases).

2. For **segmentation**, we use ADE20K [27], down-scaled to match the average resolution of CUB-200-2011. ADE20K provides 20k training and 2k validation images with dense pixel-level annotations across 150 semantic categories, offering a broad benchmark for general scene understanding.
3. For **optical flow**, we evaluate on HD1K [28], which contains 1,066 high-definition video frames (1920×1080) with dense ground truth flow fields for real-world driving scenarios. To ensure consistent input resolution with our classification setting, we resize all frames to 448×448 pixels. This benchmark stresses robustness to large displacements, motion blur, and realistic driving conditions.

All datasets are publicly available, we follow standard train/validation/test splits without additional filtering, and apply only normalization to the images, we downsample high resolution images to 448×448 for all the tasks.

Throughput Measurement: The compression penalty terms in our training objective are derived by measuring module throughput on our target device ([11]) at FP16 precision while we train our model on a single H100 [29]. For compute-bound operations, such as linear attention, we modify existing fused kernels [30] to incorporate our quantization approach. For memory-bound operations during training time, such as top- k selection and cross-entropy, we rely on off-the-shelf implementations [31]. Throughput is reported using the total data flight time, including both read and write operations.

Model Implementation: Our models are implemented in PyTorch [32] (fused operations in [33] and [34]) and trained using the AdamW optimizer [35]. We adopt a cosine learning rate decay schedule for model parameters and a cosine ramp-up schedule for the compression coefficient (γ in Equation 1), ensuring stable convergence while progressively increasing the effect of the quantization penalty.

All models are trained from random initialization, with each module initialized at 4-bit precision for quanti-

zation experiments. We report Top-1 and Top-5 accuracy for classification, mean Intersection over Union (mIoU) [36] for semantic segmentation, and average End-Point Error (EPE) [37] for optical flow, following standard evaluation practice.

4.0.2. Ablation Studies

All ablation studies to study the design of our transformer module are conducted on a model with 2.13M parameters, evaluated on the Country211 for classification [24] prediction task.

Pixel Shuffling vs Transposed Convolution: We evaluate the use of pixel shuffling [38] as an alternative to transposed convolution for the upscaling stage. Pixel shuffling is a non-learnable operation that increases spatial resolution by rearranging the input tensor. While this approach reduces computational overhead, it couples projection features across channels with different bit-depths thereby limiting the gran-

ularity of learned representations. As a result, pruning of projection convolutions is largely ineffective: channels remain coupled, and in our experiments, we observed no pruning of projection convolutions when pixel shuffling was applied.

Pointwise Projection: We investigate pointwise convolutions (1×1 kernels) as the projection stage preceding the linear attention module. Although computationally efficient, quantized 1×1 kernels tend to collapse representations toward zero at initialization, particularly when low bit-depths are assigned at initialization. To address this, we adopt 3×3 kernels for the projection stage instead of pointwise convolutions. In our experiments, we observe no degradation in training loss when comparing quantized 3×3 kernels with unquantized 1×1 kernels, indicating that slightly larger kernels maintain performance while supporting effective quantization and compression.

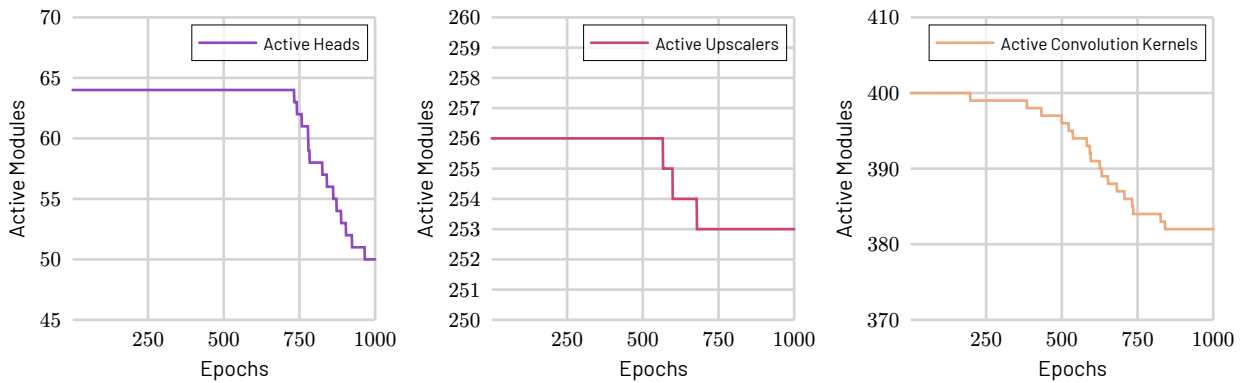


Figure 4: **Module Pruning:** The number of active modules (attention heads, upscalers, and convolution kernels) is tracked over training epochs. We observe a gradual reduction in active modules, indicating that the model prunes redundant modes of tensor as training progresses.

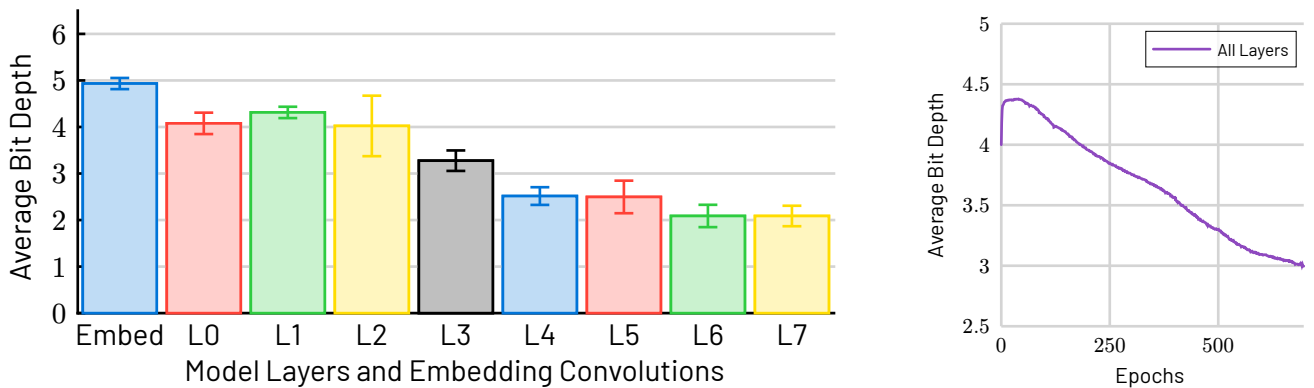


Figure 5: **Average Bit Depth Across Model Layers:** The bit depth for each module is computed as the mean across all its quantized parameters. While the embedding layer maintains higher precision than initialization (4bits), deeper layers (L) progressively converge toward lower values. The average bit depth of our modules across all layers drops to 3bits.

model	size	top1(%)	top5(%)	FLOPS (G)	throughput nvidia 4070 (images/sec)
efficientvit	0.98m	71.3	95.5	7	309
diffq	1.12m	68.1	92.3	9	281
ours	0.88m	73.4	98.0	7	338
efficientvit	2.41m	76.2	100	24	262
diffq	2.50m	71.3	100	28	276
ours	1.98m	75.8	100	24	308
efficientvit	4.81m	81.5	100	48	183
diffq	4.92m	78.1	100	54	172
ours	4.68m	83.5	100	51	230

Table 2: comparison of our models on the cub-200-2011 [23] dataset.

model	size	top1(%)	top5(%)	FLOPS (G)	throughput nvidia 4070 (images/sec)
efficientvit	2.41m	44	65	23	291
diffq	2.50m	36	41	28	270
ours	2.13m	43	65	25	324
efficientvit	4.81m	52	80	48	228
diffq	4.92m	38	65	54	213
ours	4.68m	55	81	51	294

Table 3: benchmark results on the country211 dataset [24].

model	size	miou	FLOPS (G)	throughput nvidia 4070 (images/sec)
efficientvit	4.68m	44.1	47	255
diffq	4.92m	40.1	54	241
ours	4.28m	43.7	44	282

Table 4: semantic segmentation results on the ade20k dataset [27]. where miou is mean intersection over union.

model	size	epe	FLOPS (G)	throughput nvidia 4070 (images/sec)
efficientvit	4.81m	4.8	48	191
diffq	4.92m	5.4	54	155
ours	4.68m	5.0	51	227

Table 5: optical flow estimation results on the hd1k dataset [28] where epe is end point error

Linear Attention in Parallel with Convolution Kernels:

To further improve throughput, we test running linear attention modules in parallel with convolutional kernels (similar to [12]). This design aims to offload global receptive field computation without sequential bottlenecks. However, we observe poor GPU occupancy (at inference time) when many convolution kernels are pruned.

4.1. Evaluation

For all experiments, we report inference-time size and FLOPs after module pruning of our and diffq [9] model, so results reflect the actual deployed configuration.

In **Classification** On the CUB-200-2011 dataset (Table 2), our models achieve accuracy comparable to EfficientViT across all the scales experimented.

At <1M parameters, we reach 73.4% Top-1 (vs. 71.3%) with higher throughput (338 vs. 309 images/sec). At 2M, accuracy is nearly identical (75.8% vs. 76.2%) but throughput improves (308 vs. 262). At 5M, we observe a modest gain (83.5% vs. 81.5%) while maintaining faster inference (230 vs. 183). On Country211 (Table 3), performance follows the same trend: our model matches EfficientViT at 2M parameters and improves slightly at 5M (55% vs. 52%) with higher throughput. These results show that our approach preserves accuracy while offering better efficiency.

In **semantic segmentation** on ADE20K (Table 4), our model achieves 43.7% mIoU, nearly identical to EfficientViT (44.1%) but with lower FLOPs (44 vs. 47) and higher throughput (282 vs. 255). Compared to DiffQ (40.1%), our model is both more accurate and efficient, indicating robust performance in dense prediction tasks.

For **optical flow** on HD1K (Table 5), our model attains 5.0 EPE, close to EfficientViT (4.8) and better than DiffQ (5.4). Throughput is higher (227 vs. 191), showing that our design matches the accuracy of state-of-the-art baselines while enabling faster inference.

5. Discussion

Our results demonstrate that self-compressing vision towers can closely match the accuracy of state-of-the-art lightweight backbones such as EfficientViT, while often providing modest improvements on target device in throughput and computational cost. The gains are most evident at smaller scales, where pruning and quantization remove redundant tensor channels without harming accuracy, and at larger scales, where modest accuracy improvements are coupled with faster inference. These findings support our central hypothesis: not all modules contribute equally to performance (Figure 5), and selectively allocating precision and compute across layers can yield a better balance of accuracy and efficiency.

An important aspect of our approach is that pruning is applied only at inference time. This design choice avoids irreversible pruning specially with small batch-size, where permanently removing parameters during training might lead to degraded accuracy. Since we

do not re-compile the computational graph during training, the pruned modules remain “active” (zero tensors) in the forward and backward passes, and as a result, training time across epoch remains constant. While this limits efficiency gains during optimization, it ensures that the model retains full representational capacity during learning and only discards redundant modules once training is complete, leading to stable accuracy at deployment.

6. Conclusion

We developed a novel, self-compressing vision tower that strategically prunes and quantizes all major modules, including convolutions, transposed convolutions, and linear attention, based on their computational cost and layer size. Our model’s performance on classification, segmentation, and optical flow benchmarks consistently matches the accuracy of the state-of-the-art EfficientViT, while often providing the added benefits of reduced storage and FLOPs. Furthermore, our method consistently surpasses DiffQ in both accuracy and throughput. We conclude that this constructive approach to compression not only reduces model size but also allows for the inspection of module importance across layers, facilitating more effective model design for specific target devices.

Bibliography

- [1] L. Beyer et al., "PaliGemma: A versatile 3B VLM for transfer." [Online]. Available: <https://arxiv.org/abs/2407.07726>
- [2] T. Dao, "Hybrid linear-softmax attention working very well at large scale and long-context! As we've seen with multiple models now" [Online]. Available: https://x.com/tri_dao/status/1879439184462762225
- [3] A. Gromov, K. Tirumala, H. Shapourian, P. Gloriosio, and D. A. Roberts, "The Unreasonable Ineffectiveness of the Deeper Layers." [Online]. Available: <https://arxiv.org/abs/2403.17887>
- [4] MiniMax et al., "MiniMax-01: Scaling Foundation Models with Lightning Attention." [Online]. Available: <https://arxiv.org/abs/2501.08313>
- [5] S. Mehta and M. Rastegari, "MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer," 2022. [Online]. Available: <https://arxiv.org/abs/2110.02178>
- [6] K. Wu et al., "TinyViT: Fast Pretraining Distillation for Small Vision Transformers," in *European conference on computer vision (ECCV)*, 2022.
- [7] H. Cai, J. Li, M. Hu, C. Gan, and S. Han, "EfficientViT: Multi-Scale Linear Attention for High-Resolution Dense Prediction." [Online]. Available: <https://arxiv.org/abs/2205.14756>
- [8] A. Fan, E. Grave, and A. Joulin, "Reducing Transformer Depth on Demand with Structured Dropout." [Online]. Available: <https://arxiv.org/abs/1909.11556>
- [9] A. Défossez, Y. Adi, and G. Synnaeve, "Differentiable Model Compression via Pseudo Quantization Noise." [Online]. Available: <https://arxiv.org/abs/2104.09987>
- [10] S. Cséfalvay and J. Imber, "Self-Compressing Neural Networks." [Online]. Available: <https://arxiv.org/abs/2301.13142>
- [11] NVIDIA Corporation, "NVIDIA GeForce RTX 4070 Laptop GPU." 2023.
- [12] T. Zhang, W. Xu, B. Luo, and G. Wang, "Depth-Wise Convolutions in Vision Transformers for efficient training on small datasets," *Neurocomputing*, vol. 617, p. 128998, Feb. 2025, doi: [10.1016/j.neucom.2024.128998](https://doi.org/10.1016/j.neucom.2024.128998).
- [13] M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks." [Online]. Available: <https://arxiv.org/abs/1311.2901>
- [14] P. Micikevicius et al., "FP8 Formats for Deep Learning." [Online]. Available: <https://arxiv.org/abs/2209.05433>
- [15] Y. Bengio, N. Léonard, and A. Courville, "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation." [Online]. Available: <https://arxiv.org/abs/1308.3432>
- [16] A. Vaswani et al., "Attention Is All You Need." [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [17] A. Shocher, B. Feinstein, N. Haim, and M. Irani, "From Discrete to Continuous Convolution Layers." [Online]. Available: <https://arxiv.org/abs/2006.11120>
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [19] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.

- [20] Y. Xu et al., "QT-ViT: improving linear attention in ViT with quadratic taylor expansion," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, in NIPS '24. Vancouver, BC, Canada: Curran Associates Inc., 2025.
- [21] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention." [Online]. Available: <https://arxiv.org/abs/2006.16236>
- [22] M. Beck, K. Pöppel, P. Lippe, and S. Hochreiter, "Tiled Flash Linear Attention: More Efficient Linear RNN and xLSTM Kernels." [Online]. Available: <https://arxiv.org/abs/2503.14376>
- [23] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "CUB-200-2011." CaltechDATA, Apr. 2022. doi: [10.22002/D1.20098](https://doi.org/10.22002/D1.20098).
- [24] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Country211 Dataset." 2021.
- [25] B. Thomee et al., "YFCC100M: The New Data in Multimedia Research," in *Proceedings of the 2016 ACM Conference on Multimedia Retrieval (ICMR)*, ACM, 2016, pp. 283–288. doi: [10.1145/2911996.2912006](https://doi.org/10.1145/2911996.2912006).
- [26] Wikimedia Commons contributors, "Wikimedia Commons, the free media repository." [Online]. Available: <https://commons.wikimedia.org/>
- [27] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Scene Parsing through ADE20K Dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 5122–5130. doi: [10.1109/CVPR.2017.544](https://doi.org/10.1109/CVPR.2017.544).
- [28] D. Kondermann et al., "The HCI Benchmark Suite: Stereo and Flow Ground Truth with Uncertainties for Urban Autonomous Driving," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2016, pp. 19–28. doi: [10.1109/CVPRW.2016.121](https://doi.org/10.1109/CVPRW.2016.121).
- [29] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU." 2022.
- [30] S. Yang and Y. Zhang, "FLA: A Triton-Based Library for Hardware-Efficient Implementations of Linear Attention Mechanism." [Online]. Available: <https://github.com/fla-org/flash-linear-attention>
- [31] Dao-AILab, "QuACK: A Quirky Assortment of CuTe Kernels." 2025.
- [32] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.
- [33] N. Corporation, "CuTe: CUDA Tensor Expressions." 2025.
- [34] J. Jiang et al., "Triton: An Intermediate Language and Compiler for Optimizing Deep Learning on GPUs," in *Advances in Neural Information Processing Systems (NeurIPS) 35*, 2022. [Online]. Available: <https://openreview.net/forum?id=7phvJ2kvEY>
- [35] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [36] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010, doi: [10.1007/s11263-009-0275-4](https://doi.org/10.1007/s11263-009-0275-4).
- [37] S. Baker, D. Scharstein, J. Lewis, S. Roth, M. J. Black, and R. Szeliski, "A Database and Evaluation Methodology for Optical Flow," in *International Conference on Computer Vision (ICCV)*, 2011, pp. 233–236. doi: [10.1109/ICCV.2011.6126252](https://doi.org/10.1109/ICCV.2011.6126252).

- [38] W. Shi et al., "Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network." [Online]. Available: <https://arxiv.org/abs/1609.05158>

A. Report Checklist /

This checklist was created by Chang Sun for Bachelor's and Master's students at the Department of Advanced Computing Sciences, Maastricht University. It supports writing a complete, clear, and scientifically sound thesis. Students are encouraged to share this checklist with supervisors and highlight the items they have addressed.

A.A. Introduction

1. **Problem Statement:** Clearly define the key problem your thesis addresses. [research_question](#) 
2. **Significance:** Explain why this problem is important and urgent. [significance](#) 
3. **Research Challenges:** Identify why the problem has not been solved yet. 
4. **Brief Description of This Work:** Summarize your approach and how it differs from existing methods. [desc](#) 
5. **(Optional) Illustrative Diagram:** Include a simple, easy-to-understand diagram. 
6. **Key Contributions:** List and highlight the innovative contributions. [contribution](#) 
7. **Thesis Structure:** Briefly describe what each chapter covers. 

A.B. Related Work

1. Summarize state-of-the-art methods relevant to your thesis. 
2. Discuss strengths and limitations. 
3. Identify gaps in current work. 
4. Explain how your thesis addresses these gaps. 

A.C. Methodology

1. **Approach Overview:** High-level view with a well-labeled diagram 
2. **Components:** Explain each component in separate subsections. 
3. **Implementation Details:**
 1. Method implementation 
 2. Architecture choices, algorithms, and software 
 3. Code must be open-source and documented on GitHub. 

A.D. Datasets

1. **Data Sources & Collection:**
 1. What, where, when, and how data was collected 
 2. Flowchart: source → filtering → final dataset 
 3. Inclusion/exclusion criteria and counts 
 4. Provide access details in footnotes or appendix 
2. **Dataset Description:**
 1. Size, distributions, missing values 
 2. Preprocessing steps 

A.E. Experiments

1. **Design:**
 1. Dataset splits (training/test/validation) 
 2. Which experiments are run, which parameters tested 

we do not provide details on hyper-parameter testing (such as initializing module bit-depth).
2. **Settings:**
 1. Hardware (GPU/CPU model) 
 2. Epochs, batch size, learning rate, etc. 
3. **(Include figures/flowcharts when design is complex.)** 

A.F. Evaluation Methodology

1. Describe performance evaluation strategy. ✓
2. List and define metrics (e.g., accuracy, precision). ✓
3. Include equations and implementation notes if possible. ✓

A.G. Results

1. Use tables and figures to report results clearly. ✓
2. Highlight key numbers readers should focus on. ✓
3. **Statistical Significance:**
 1. Show deviations, error bars, or other significance metrics. ✓
4. Objectively compare models, parameters—avoid interpretations here. ✓

A.H. Discussion

1. Interpret findings and patterns. ⚠
2. Reflect on unexpected trends. ⚠
3. **Limitations:** What didn't work, assumptions, robustness. ⚠
4. **Future Work:** Propose directions for improvement. ✓
5. **(Optional)** Discuss scientific and societal impacts.

A.I. Conclusion

1. Concise summary of your thesis. ⚠
2. Highlight discoveries and findings. ⚠
3. Share lessons learned. ⚠
4. **(If not in Discussion)** mention future work directions.

A.J. References

1. All references must include DOI or persistent IDs. ✓
2. Prefer peer-reviewed papers. Preprints (e.g., arXiv) are allowed but disclose their proportion ⚠. ✓

A.K. Chapter Organization

Sections 4–8 may be structured in one of the following ways:

1. Separate each as an individual chapter.
2. Combine:
 1. **Datasets + Experiments + Evaluation** in one chapter;
 2. **Results + Discussion** in another.
3. Or:
 1. **Dataset** in one chapter;
 2. **Experiments, Evaluation, Results** in another;
 3. **Discussion** separately.
4. Other logical groupings are allowed if all parts are clearly covered.

B. Defense Presentation Checklist

1. What is the problem?
2. Why is it significant?
3. Related work
4. Research questions
5. Datasets, study design, and experiments

6. Results and how they answer the research questions
7. List of key contributions
8. Future work directions